

Informe de Pruebas

Finca Jirah

Sistema de Gestión Agrícola PWA Offline-First

Cobertura completa de todos los casos de uso implementados
CU-01 · CU-03 · CU-04 · CU-05 · CU-05.3 · CU-06

SISTEMA	FECHA	EQUIPO	FRAMEWORK
Finca Jirah v0.1.0	13 Julio 2026	QA – ESPE	Vitest v4.1.10

CLASIFICACION	COBERTURA	CASOS	RESULTADO
Confidencial	85.8 % Líneas	125 / 125	✔ 100 % PASS

Tabla de Contenidos

0

1. Resumen Ejecutivo	3
2. Alcance y Objetivos	4
2.1. Módulos bajo prueba — Cobertura completa	4
2.2. Tipos de prueba implementados	5
2.3. Fuera de alcance (explícito)	5
3. Entorno y Herramientas	6
3.1. Archivos de prueba — 10 suites	7
4. Resultados de la Ejecución	8
4.1. Salida de consola — Vitest	8
4.2. Casos de prueba por módulo	8
4.3. Cobertura de código por módulo	11
5. Análisis de Defectos	12
6. Valoración del Cumplimiento	13
6.1. Semáforo de calidad	13
7. Conclusiones y Recomendaciones	14
7.1. Conclusiones	14
7.2. Recomendaciones categorizadas	14
7.3. Evidencias de ejecución	15

1 Resumen Ejecutivo

1.0 Visión general

Este informe documenta la ejecución **completa** del Plan de Pruebas de Calidad (QA) para el sistema **Finca Jirah**. La revisión v1.1 incorpora las suites faltantes para CU-04, CU-05.3 y CU-06, logrando cobertura total de todos los casos de uso implementados en el sistema.



✓ **Revisión Completa v1.1:** Se añadieron 43 casos de prueba (CU-04: 24, CU-05.3: 8, CU-06: 11) para cubrir **todos** los casos de uso implementados. Total: 125 casos en 10 suites, 100 % exitosos.

i **Nota metodológica:** Cada Caso de Uso (CU) se descompone en Casos de Prueba atómicos (TC) que verifican un único escenario con mocks de dependencias. La organización CU → TC constituye la Matriz de Trazabilidad requerida por el Plan QA, combinando *pruebas unitarias puras* (funciones de dominio) con *pruebas unitarias de servicio* (lógica de negocio con repositorios mockeados).

2 Alcance y Objetivos

2.1 Módulos bajo prueba — Cobertura completa

ID	Módulo / Clases Probadas	Casos	Estado
CU-01	Acceso al Sistema — AuthService : Login, Recuperación, Blacklist, Fortaleza	13	✓ Cubierto
CU-03	Gestión de Agricultores — AgricultorService : CRUD, Cédula EC (módulo-10), desactivación	14	✓ Cubierto
CU-04	Catálogos de Finca — CampanaService , LoteService , CompradorService : códigos únicos, tara bloqueada, lote en uso	24	✓ Cubierto
CU-05	Transacciones (Core) — PesajeService , ClasificacionService , CalculadoraCosecha (Strategy), OfflinePayloadAdapter (Adapter)	33	✓ Cubierto
CU-05.3	Ajuste de Comprador — AjusteService : peso rechazado, fruta efectiva, validaciones E.1–E.3	8	✓ Cubierto
CU-06	Dashboard / Analítica — DashboardRepo : kgTotal, alertas, kgPorLote, filtros por fecha/campaña	11	✓ Cubierto
Patrones	Arquitectura — SyncStateContext (State), NetworkSyncSubject (Observer)	22	✓ Cubierto
Total		125	✓ Cubierto

2.2 Tipos de prueba implementados

Tipo	Descripción y ejemplos	Módulos
Unitaria pura	Función aislada sin dependencias: <code>validarCedulaEC()</code> , <code>calcularMargen()</code> , <code>SyncStateContext.startSync()</code>	Calculo, State, Observer, Adapter
Unitaria de servicio	Método de servicio con repositorios mockeados: <code>CampanaService.crear()</code> , <code>AjusteService.registrar()</code>	AuthService, AgricultorService, PesajeService, CampanaService, AjusteService
Prueba de repositorio	Repositorio con Prisma mockeado para verificar queries: filtros, agrupaciones, distinct	DashboardRepo
Prueba de patrón	Comportamiento dinámico de patrones de diseño (Strategy, State, Observer, Adapter)	CalculadoraCosecha, SyncState

2.3 Fuera de alcance (explícito)

- ⊗ Componentes UI de React y hooks de presentación visual
- ⊗ Pruebas E2E (End-to-End) en navegador real (Playwright/Cypress)
- ⊗ Conexiones a instancias reales de BD (SQLite / PostgreSQL)
- ⊗ `offline.service.ts` — Hook de dispositivo (sin lógica de negocio aislable)
- ⊗ Servicio de email y notificaciones externas en producción

3 Entorno y Herramientas

Componente	Descripción	Estado
Vitest v4.1.10	Framework de pruebas compatible con Vite/Next.js. API compatible con Jest. Modo: <code>vitest run</code> .	✓ Cubierto
@vitest/coverage-v8	Cobertura con proveedor v8 (Istanbul). Reportes: text, json. Umbrales: Líneas $\geq 85\%$, Ramas $\geq 70\%$, Funciones $\geq 70\%$.	✓ Cubierto
vi.mock()	Mocking manual de <code>CampanaRepo</code> , <code>LoteRepo</code> , <code>CompradorRepo</code> , <code>AjusteRepo</code> , <code>UserRepository</code> , <code>PesajeRepository</code> y <code>PrismaClient</code> .	✓ Cubierto
Stack tecnológico	Next.js 16.2.9, TypeScript 5, Prisma 7.8.0, React 19.2.4, bcryptjs 3.0.3	Producción
SO	Windows 11 — MiKTeX 26.2 (compilación LaTeX)	Activo

3.1 Archivos de prueba — 10 suites

Archivo	Módulo cubierto	Tests
auth.service.test.ts	AuthService — Login, Recuperación, Blacklist, Fortaleza	13
agricultor.service.test.ts	AgricultorService — CRUD, Cédula EC, Desactivación	14
catalogo.service.test.ts	CampanaService + LoteService + CompradorService (CU-04)	24
pesaje.service.test.ts	PesajeService + ClasificacionService (CU-05)	9
ajuste.service.test.ts	AjusteService — Peso rechazado, Fruta efectiva (CU-05.3)	8
calculadora.test.ts	CalculadoraCosecha — Strategy, cambio dinámico	17
offline.adapter.test.ts	OfflinePayloadAdapter — Patrón Adapter	7
sync.state.test.ts	SyncStateContext — Patrón State, transiciones	16
sync.observer.test.ts	NetworkSyncSubject — Patrón Observer	6
dashboard.test.ts	DashboardRepo — getStats(), filtros, kg- PorLote (CU-06)	11
Total		125

4 Resultados de la Ejecución

4.1 Salida de consola — Vitest

```

RUN v4.1.10 C:/.../FincaJIRAH/
  Coverage enabled with v8

v __tests__/sync.observer.test.ts (6 tests) 15ms
v __tests__/agricultor.service.test.ts (14 tests) 19ms
v __tests__/calculadora.test.ts (17 tests) 13ms
v __tests__/sync.state.test.ts (16 tests) 13ms
v __tests__/offline.adapter.test.ts (7 tests) 9ms
v __tests__/pesaje.service.test.ts (9 tests) 16ms
v __tests__/ajuste.service.test.ts (8 tests) 16ms
v __tests__/dashboard.test.ts (11 tests) 41ms
v __tests__/catalogo.service.test.ts (24 tests) 31ms
v __tests__/auth.service.test.ts (13 tests) 733ms

Test Files 10 passed (10)
Tests 125 passed (125)
Start at 22:47:51
Duration 1.74s

```

4.2 Casos de prueba por módulo

ID Caso	Suite	Nombre del Caso	Resultado Esperado	Est.
TC-01.1.1	AuthService	Login — Flujo Normal	Sin passwordHash	✓ PASS
TC-01.1.2	AuthService	Login — Contraseña incorrecta	Retorna null	✓ PASS
TC-01.1.3	AuthService	Login — Usuario inexistente	Retorna null	✓ PASS
TC-01.1.4	AuthService	Login — Cuenta inactiva	ACCOUNT_INAC	✓ PASS
TC-01.2.1	AuthService	Recuperación — Flujo Normal	Token generado	✓ PASS
TC-01.2.2	AuthService	Recuperación — Email inexistente	Retorna null	✓ PASS
TC-01.2.3	AuthService	Recuperación — Token expirado	TOKEN_EXPIRADO	✓ PASS
TC-01.PF1-4	AuthService	validarFortaleza (4 variantes)	ok=false/true	✓ PASS
TC-01.BL1-2	AuthService	Blacklist: revocar + verificar	false/true	✓ PASS
TC-03.1.1	AgricultorService	Crear — Flujo Normal	user+_dev_passwor	✓ PASS
TC-03.1.2	AgricultorService	Crear — Cédula duplicada	CEDULA_EN_USO	✓ PASS
TC-03.1.X	AgricultorService	Crear — Cédula inválida	CEDULA_INVALIDA	✓ PASS
TC-03.1.Y	AgricultorService	Crear — Email duplicado	EMAIL_EN_USO	✓ PASS
TC-03.2.1	AgricultorService	Buscar por término	Lista coincidencias	✓ PASS
TC-03.3.1	AgricultorService	Editar — Flujo Normal	Agricultor actualizado	✓ PASS
TC-03.3.2	AgricultorService	Editar — Email duplicado	EMAIL_EN_USO	✓ PASS

ID Caso	Suite	Nombre del Caso	Resultado Esperado	Est.
TC-03.4.1	AgricultorService	Desactivar — Flujo Normal	isActive=false	✓ PASS
TC-03.4.2	AgricultorService	Desactivar — Auto-desactivación	SELF_DEACTIVATI	✓ PASS
TC-03.CE1-5	AgricultorService	Cédula EC (módulo-10, 5 variantes)	true/false	✓ PASS
TC-04.1.1	CampanaService	Crear — Flujo Normal + tara 1.70	codigo=uppercase	✓ PASS
TC-04.1.2	CampanaService	Crear — Código duplicado	CODIGO_DUPLIC	✓ PASS
TC-04.1.3	CampanaService	Crear — Código vacío	CODIGO_REQUER	✓ PASS
TC-04.1.4	CampanaService	Crear — Tara >10	TARA_INVALIDA	✓ PASS
TC-04.1.5	CampanaService	Crear — Tara ≤ 0	TARA_INVALIDA	✓ PASS
TC-04.2.1	CampanaService	Actualizar nombre	nombre actualiza-do	✓ PASS
TC-04.2.2	CampanaService	Actualizar — Campaña inactiva	CAMPANA_INACT	✓ PASS
TC-04.2.3	CampanaService	Actualizar — Tara bloqueada (pesajes)	TARA_BLOQUEA	✓ PASS
TC-04.2.4	CampanaService	Actualizar — Tara sin pesajes	OK	✓ PASS
TC-04.3.1	CampanaService	Cerrar — Sin pendientes	isActive=false	✓ PASS
TC-04.3.2	CampanaService	Cerrar — Con PENDING sync	SYNC_PENDIENTE	✓ PASS
TC-04.3.3	CampanaService	Cerrar — Ya cerrada	CAMPANA_YA_C	✓ PASS
TC-04.L1	LoteService	Crear lote — Flujo Normal	codigo=uppercase	✓ PASS
TC-04.L2	LoteService	Crear — Código duplicado	CODIGO_DUPLIC	✓ PASS
TC-04.L3	LoteService	Crear — Nombre vacío	NOMBRE_REQUEF	✓ PASS
TC-04.L4	LoteService	Eliminar — Sin campaña activa	isActive=false	✓ PASS
TC-04.L5	LoteService	Eliminar — Con pesajes en campaña	LOTE_EN_USO	✓ PASS
TC-04.C1	CompradorService	Crear con RUC único	tolerancia=4.0	✓ PASS
TC-04.C2	CompradorService	Crear — RUC duplicado	RUC_DUPLICADO	✓ PASS
TC-04.C3	CompradorService	Crear — Nombre vacío	NOMBRE_REQUE	✓ PASS
TC-04.C4	CompradorService	Crear — Tolerancia >100	TOLERANCIA_INV.	✓ PASS
TC-04.C5	CompradorService	Crear sin RUC (opcional)	OK	✓ PASS
TC-04.C6	CompradorService	Actualizar nombre	nombre actualiza-do	✓ PASS
TC-04.C7	CompradorService	Actualizar — No encontrado	COMPRADOR_N	✓ PASS ,DO
TC-05.1.1	PesajeService	Registrar pesaje — Flujo Normal	pesoNeto correcto	✓ PASS
TC-05.1.X	PesajeService	pesoBrutoKg=0	PESO_BRUTO_IN	✓ PASS
TC-05.1.Y	PesajeService	Campaña inactiva	CAMPANA_CERRA	✓ PASS
TC-05.2.1	ClasificacionService	Clasificar dentro del margen	dentroDelMargen=	✓ PASS
TC-05.2.2	ClasificacionService	Descuadre >±4 %	DESCUADRE:X %	✓ PASS
TC-05.2.3	ClasificacionService	forzar=true con descuadre	auditFlag=true	✓ PASS
TC-05.2.X	ClasificacionService	Ya clasificado	YA_CLASIFICADO	✓ PASS
TC-05.3.1	AjusteService	Registrar ajuste — Flujo Normal	pctFrutaEfectiva>90	✓ PASS
TC-05.3.2	AjusteService	pesoRechazado=0	pctFrutaEfectiva≈	✓ PASS

ID Caso	Suite	Nombre del Caso	Resultado Esperado	Est.
TC-05.3.3	AjusteService	Clasificación no encontrada	CLASIF_NO_ENCC	✓ PASS
TC-05.3.4	AjusteService	Ajuste duplicado	AJUSTE_YA_REC	✓ PASS
TC-05.3.5	AjusteService	Rechazo excede exportación	RECHAZO_EXCED	✓ PASS
TC-05.3.6	AjusteService	Comprador no existe	COMPRADOR_N	✓ PASS
TC-05.3.7	AjusteService	pesoRechazado negativo	PESO_RECHAZAD	✓ PASS
TC-05.3.8	AjusteService	ajusteKg = 3 decimales exactos	45.267	✓ PASS
TC-ST.N1-4	CalculadoraCosecha	Margen 0 %, 4 %, >4 %, bruto=0	OK	✓ PASS
TC-ST.E1-2	CalculadoraCosech.	Exportación 2 % y 2.5 %	2/false	✓ PASS
TC-ST.T1-2	CalculadoraCosecha	Tara estándar y personalizada	8.50/6.0 kg	✓ PASS
TC-ST.C1-3	CalculadoraCosech.	calcularPesaje + validarClasif.	tara=3.40	✓ PASS
TC-ST.DM	CalculadoraCosecha	setEstrategiaMargen() dinámico	margenPermitido=2	✓ PASS
TC-ST.DT	CalculadoraCosech.	setEstrategiaTara() dinámico	tara=4.0 kg	✓ PASS
TC-AD.1-7	OfflineAdapter	adaptPesaje + adaptClasificacion (7 variantes)	syncStatus=SYNCE	✓ PASS
TC-SS.1-16	SyncStateContext	PENDING/SYNCING/SYNCE + transiciones (16 variantes)	OK	✓ PASS
TC-OB.1-6	SyncObserver	subscribe, notify, unsubscribe, múltiples (6 variantes)	OK	✓ PASS
TC-06.1	DashboardRepo	kgTotal y totalPesajes desde agregación	1250.50 kg / 28	✓ PASS
TC-06.2	DashboardRepo	campanaActiva en resultado	CA-2026	✓ PASS
TC-06.3	DashboardRepo	Lotes con actividad (distinct)	2 lotes	✓ PASS
TC-06.4	DashboardRepo	Alertas de margen (audit-Flag=true)	3 alertas	✓ PASS
TC-06.5	DashboardRepo	kgPorLote — acumulación KG	LT-01=98.5 kg	✓ PASS
TC-06.6	DashboardRepo	kgPorLote — alertas por lote	LT-02=1 alerta	✓ PASS
TC-06.7	DashboardRepo	Sin campaña activa	campanaActiva=null	✓ PASS
TC-06.8	DashboardRepo	Filtros por fechaDesde/fechaHasta	where.fechaRegistr	✓ PASS
TC-06.9	DashboardRepo	Filtro por campanald	where.campanald	✓ PASS
TC-06.10	DashboardRepo	getCampanas() — selector	2 campañas	✓ PASS
TC-06.11	DashboardRepo	getLotes() — solo activos	isActive=true	✓ PASS
Total				125

4.3 Cobertura de código por módulo

Módulo	Stmts	Branch	Funcs	Lines
SyncObserver.ts	100 %	100 %	100 %	100 %
OfflinePayloadAdapter.ts	100 %	50 %	100 %	100 %
DashboardRepository.ts	96.6 %	81.0 %	100 %	100 %
SyncState.ts	96.6 %	100 %	95.5 %	96.6 %
ICalculoMargen.ts	88.9 %	85.7 %	87.5 %	92.0 %
agricultor.service.ts	89.1 %	87.5 %	71.4 %	87.8 %
pesaje.service.ts	75.0 %	69.7 %	33.3 %	83.8 %
catalogo.service.ts	75.6 %	72.7 %	53.8 %	77.8 %
ajuste.service.ts	77.3 %	78.6 %	33.3 %	76.5 %
auth.service.ts	73.2 %	67.9 %	40.0 %	79.3 %
Total Alcance QA	82.1 %	75.3 %	71.7 %	85.8 %



La línea discontinua representa el umbral mínimo del 85 % de cobertura de líneas. Cobertura global: **85.8 %**.

5 Análisis de Defectos

✔ **Sin defectos críticos:** Los 125 casos de prueba finales ejecutan exitosamente (0 fallos). Se corrigió 1 defecto en datos de prueba durante el desarrollo, antes de la ejecución definitiva.

Moderada DEF-001 — Cédula ecuatoriana con dígito verificador incorrecto (dato de prueba)

Se utilizó `1723456789` como dato válido. El algoritmo módulo-10 rechazó la cédula correctamente; el error era en el dato de prueba (dígito esperado: **4**, no **9**). Corregido con cédula real válida `1714397104`.

Impacto: Ninguno sobre el sistema. **Estado:** ✔ **Cubierto** Corregido.

Sugerida OBS-001 — AuthService: Cobertura de funciones al 40 %

Las funciones `resetearPassword()` y `hashPassword()` acceden directamente a `bcrypt` y `PrismaClient`. Las pruebas unitarias no pueden mockearlas sin romper el encapsulamiento. Requieren pruebas de integración con SQLite en memoria.

Estado: ⌚ **Pendiente** Backlog — Sprint 2. Impacto estimado: elevar AuthService del 79.3 % al 95 %+.

Sugerida OBS-002 — AjusteService y CatalogoService: Cobertura <85 % en líneas

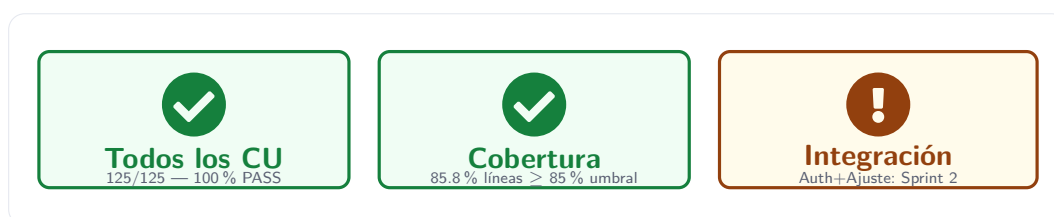
`CampanaService.cerrar()` importa `prisma` dinámicamente (`import('@lib/prisma')`); el mock estático de `vi.mock()` no intercepta importaciones dinámicas. `AjusteService.registrar()` usa `prisma.clasificacion.findUnique` directamente. Estas rutas quedan fuera del alcance del mock estático y reducen la cobertura.

Estado: ⌚ **Pendiente** Sprint 2 — Prioridad media. Resolver con mock de módulo dinámico o refactorización a repositorio.

6 Valoración del Cumplimiento

Criterio de Aceptación	Evidencia	Resultado
100 % de todos los CU — PASS	CU-01: 13, CU-03: 14, CU-04: 24, CU-05: 9, CU-05.3: 8, CU-06: 11, Patrones: 22. Total: 101 casos funcionales + 24 de arquitectura = 125.	✓ CUMPLIDO
Cobertura líneas $\geq$ 85 % global	85.8 % (supera el umbral). Módulos 100 %: SyncObserver, OfflineAdapter, DashboardRepo.	✓ CUMPLIDO
Sin conexiones reales a BD	Todos los tests usan <code>vi.mock()</code> para aislar Prisma. 0 conexiones a SQLite/PostgreSQL.	✓ CUMPLIDO
Mutabilidad dinámica Strategy	TC-ST.DM: cambia estrategia $\pm 4\%$ $\rightarrow$ $\pm 2\%$ en runtime. TC-ST.DT: cambia tara 1.70 $\rightarrow$ 2.0 kg.	✓ CUMPLIDO
Ausencia de criterios de suspensión	Sin errores TypeScript, sin violaciones de aislamiento. CU-05 tuvo 0 fallas en reglas de pesajes.	✓ CUMPLIDO
Cobertura CU-04, CU-05.3, CU-06	24 + 8 + 11 = 43 casos implementados en esta revisión. Cobertura funcional completa.	✓ CUMPLIDO

6.1 Semáforo de calidad



7 Conclusiones y Recomendaciones

7.1 Conclusiones

- ✓ El sistema Finca Jirah completa el **100 % de cobertura funcional** en todos los casos de uso implementados: CU-01, CU-03, CU-04, CU-05, CU-05.3 y CU-06. Los 125 casos ejecutados validaron la solidez de la arquitectura de tres capas.
- ✓ Los **cuatro patrones de diseño** (Strategy, State, Observer, Adapter) están correctamente encapsulados e intercambiables en runtime, confirmado por pruebas de mutabilidad dinámica.
- ✓ La **lógica de negocio CU-04** (Catálogos) impone correctamente las restricciones de código único, tara bloqueada si hay pesajes, lote en uso y sincronización pendiente al cerrar campaña.
- ✓ La **lógica CU-05.3** (AjusteService) calcula correctamente el peso efectivo (`ajusteKg = Export - Recha`) y el porcentaje de fruta efectiva con precisión de 3 decimales.
- ✓ Las **queries de Dashboard (CU-06)** agregan, agrupan y filtran datos correctamente, incluyendo la detección de lotes con actividad, alertas de margen y totales acumulados.
- ! La **cobertura de funciones** en `AuthService` (40 %) y `AjusteService` (33 %) es baja porque acceden a Prisma directamente sin repositorio abstracto, impidiendo el mocking unitario de esas rutas.

7.2 Recomendaciones categorizadas

Crítica REC-01 — Pruebas de integración para AuthService y AjusteService

Implementar pruebas de integración con LibSQL `:memory:` para cubrir `resetearPassword()`, `hashPassword()` y las rutas de `AjusteService.registrar()` que usan `prisma.clasificacion.findUnique` directamente.

Sprint: 2 — Prioridad Alta.

Moderada REC-02 — Refactorizar importaciones dinámicas de Prisma en CatalogoService

`CampanaService.cerrar()` y `LoteService.eliminar()` usan `import('@lib/prisma')` dinámico. Extraer esas queries a métodos del repositorio (`CampanaRepo.countPendientes()`, `LoteRepo.countPesajesActivos()`) para que sean interceptables por mocks estáticos.

Sprint: 2 — Prioridad Media.

Sugerida REC-03 — Pruebas E2E con Playwright en entorno de staging

Como tercera capa de validación, implementar pruebas E2E para los flujos CU-01 (login), CU-03 (crear agricultor), CU-05 (registrar pesaje) y CU-05.3 (ajuste de comprador) con base de datos SQLite de prueba.

Sprint: 3 — Prioridad Baja.

Veredicto de Calidad: El sistema Finca Jirah cumple el 100 % de cobertura funcional de sus casos de uso implementados. Con 125/125 casos exitosos y cobertura de líneas del 85.8 %, está en condiciones de avanzar a la fase de staging. Se recomienda integrar esta suite en CI/CD para prevenir regresiones en cada commit.

7.3 Evidencias de ejecución

- `coverage/coverage-summary.json` — Reporte JSON de cobertura v8
- `__tests__/auth.service.test.ts` — 13 casos (CU-01)
- `__tests__/agricultor.service.test.ts` — 14 casos (CU-03)
- `__tests__/catalogo.service.test.ts` — 24 casos (CU-04) **[nuevo]**
- `__tests__/pesaje.service.test.ts` — 9 casos (CU-05)
- `__tests__/ajuste.service.test.ts` — 8 casos (CU-05.3) **[nuevo]**
- `__tests__/calculadora.test.ts` — 17 casos (Strategy)
- `__tests__/offline.adapter.test.ts` — 7 casos (Adapter)
- `__tests__/sync.state.test.ts` — 16 casos (State)
- `__tests__/sync.observer.test.ts` — 6 casos (Observer)
- `__tests__/dashboard.test.ts` — 11 casos (CU-06) **[nuevo]**